

TP1 PROGRAMMATION EMBARQUEE

Réseau multimaître CAN

Ce projet se concentre sur l'analyse et la configuration de la communication au sein d'un réseau CAN utilisé pour contrôler divers modules dans un système embarqué. Le système embarqué est constitué de diverses cartes qui communiquent à l'aide du protocole CAN. [cite : 1, 2, 3, 4, 5]

Questions Théoriques

1.1 Contexte général

1. Quel est le rôle de la carte processeur basée sur le 68332 dans le réseau CAN ?
2. Pourquoi une carte contrôleur (CAN) est-elle nécessaire dans ce système ?
3. Quels sont les avantages de l'utilisation du protocole CAN pour la communication entre le microcontrôleur et les modules périphériques dans cette application ?

1.2 Réseau CAN

1. Quelles sont les caractéristiques du réseau CAN utilisé dans ce système embarqué ? Donnez des exemples des types de modules qui y sont connectés (par exemple, sortie de puissance, entrée numérique).
2. Pourquoi le réseau CAN est-il adapté à la transmission de données entre le microcontrôleur et les divers modules de contrôle dans un système d'éclairage de véhicule ?

1.3 Sécurité et Fiabilité

- Quelles mesures pourraient être mises en œuvre dans le logiciel pour garantir l'intégrité et la validité des messages CAN, en particulier en

ce qui concerne les données envoyées pour contrôler les lumières ?

- Comment le système peut-il être conçu pour gérer la défaillance d'un module CAN ou une perturbation de la communication sur le bus CAN ?

1.4 Configuration Réseau

- Comment les identificateurs CAN sont-ils attribués aux différents modules (par exemple, le module de puissance à 4 sorties) dans le système fourni ? (Faire référence aux tableaux d'identificateurs mentionnés dans le document)
- Quelles fonctions logicielles (de Mtr_CAN.h et Mtr_CAN.C) sont utilisées pour transmettre et recevoir des trames CAN ?

1.5 Format des Trames

- Comment une trame CAN est-elle structurée pour envoyer des commandes à un module spécifique afin de contrôler ses sorties (par exemple, allumer/éteindre une lampe) ? (Faire référence aux concepts de « Input message » et « Write Register »)
- Donnez un exemple de trame CAN utilisée pour :
 - Inclure l'identificateur, la longueur des données et les champs de données.
 - Contrôler l'état d'une lampe spécifique dans l'ensemble « Feux arrière droit ».

1.6 Simulation et Débogage

- Quelles méthodes sont disponibles dans l'environnement de développement du système embarqué pour surveiller le trafic du bus CAN et inspecter les trames transmises/reçues ?
- Comment le logiciel peut-il être utilisé pour simuler des erreurs dans la communication CAN (par exemple, corruption de données, messages perdus) afin de tester la gestion des erreurs du système ?

1.7 Extensions Possibles

- Comment pourriez-vous mettre en œuvre un mécanisme logiciel pour filtrer ou valider les messages CAN entrants en fonction de leurs identificateurs ou du contenu des données ?

- Proposez une méthode pour garantir qu'un module (par exemple, un module de commande de lumière) ne réponde qu'aux commandes provenant d'une source autorisée.

1.8 Questions d'analyse

1.8.1 Cas d'étude

- Si un module d'éclairage cesse de répondre aux commandes CAN, quelles mesures prendriez-vous pour diagnostiquer le problème à l'aide des outils logiciels et matériels disponibles ?
- Comment le logiciel peut-il être optimisé pour minimiser la charge du bus CAN et assurer la livraison en temps opportun des messages de contrôle critiques ?

1.9 Intégration du protocole CAN

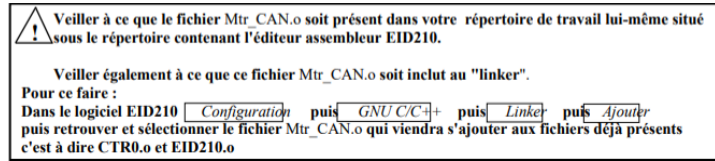
- Pourquoi le protocole CAN est-il bien adapté à la communication dans les systèmes embarqués automobiles comme celui décrit ?
- Quelles sont les limitations potentielles du protocole CAN dans cette application, et existe-t-il des technologies de communication alternatives ou complémentaires qui pourraient être envisagées ?

2.1 Sujet :

Vous devez répondre aux questions 1 à 5 en faisant le lien avec les questions discutées pendant le Tp2.

Objectifs :

- Comprendre et utiliser les structures de données spécifiques proposées.
- Comprendre et utiliser les fonctions spécifiques proposées.
- Définir puis envoyer une trame de données à un module CAN destinataire.
- Rendre accessible une adresse donnée.
- Tester si une trame a été reçue.
- Visualiser sur l'écran les trames reçues ainsi que les trame envoyées.



Cahier des charges :

- Au départ toutes les lampes du bloc « Feux arrière droit » sont éteintes.
- On souhaite réaliser la fonction « chenillard » avec les 4 lampes du bloc (A intervalles de temps réguliers, on éteint la lampe précédemment allumée et on allume la suivante).
- Les trames reçues ou envoyées sur le bus CAN sont affichées.
- La temporisation est de type « logiciel » (comptage du nombre de passage dans la boucle principale).
- Le programme devra permettre, après un minimum de modifications, de réaliser la même fonction avec les blocs « Feux arrière gauche », « Feux avant gauche » et « Feux avant droit ».
- Trois taches seront mises en œuvres.
 - La première traite le chenillard.
 - La seconde envoie la trame a destination du bloc optique sélectionné.
 - La troisième lit et affiche la trame reçue.

Ci-dessous les cartes que vous trouverez dans la maquette des véhicules multiplexés.

Réseau CAN

- Carte Processeur autour du 68332
- Carte contrôleur CAN (pCAN)
- Carte CAN 8 entrées TOR
- Carte CAN 4 sorties de puissance
- Carte CAN Asservissement



FIGURE 1 – Cartes du véhicule multiplexé

Document 1A CARTES EMBARQUEES

Matériels et logiciels nécessaires :

- Micro ordinateur de type PC sous Windows 95 ou ultérieur
- Logiciel Editeur-Assembleur-Debogueur
- Si programmation en C, Compilateur GNU C/C++ Réf : EID210100
- Carte processeur 16/32 bits à microcontrôleur 68332 avec noyau temps réel intégré et son environnement logiciel (Editeur-Cross Assembleur-Debogueur) Réf : EID210000
- Carte réseau CAN PC/104 de chez ATON SYSTEMES Ref : EID004000
- Réseau CAN avec modules 4 sorties de puissance destiné aux feux Réf : EID051000
- Câble de liaison USB, ou à défaut câble RS232, Réf : EGD000003
- Alimentation AC/AC 8V, 1A Ref : EGD000001
- Alimentation 12V pour l'alimentation des modules CAN (réseau « énergie »).

2.2 Les fonctions spécifiques

Des fonctions spécifiques à l'accès au réseau CAN via la carte CAN-PC104 de chez « ATON SYSTEMES ».

Les prototypes de ces fonctions sont définis dans le fichier Mtr_CAN.h et leurs définitions dans le fichier Mtr_CAN.C

S'il ne l'est pas déjà, ce fichier doit être compilé et son résultat de compilation Mtr_CAN.o doit être inclut en tant que fichier à « Linker » avec le fichier application.

Veiller à ce que le fichier Mtr_CAN.o soit présent dans votre répertoire de travail lui-même situé sous le répertoire contenant l'éditeur assembleur EID210.

Veiller également à ce que ce fichier Mtr_CAN.o soit inclut au « linker ».

Pour ce faire :

Dans le logiciel EID210

Configuration puis GNU C/C++ puis Linker puis Ajouter

puis retrouver et sélectionner le fichier Mtr_CAN.o qui viendra s'ajouter aux fichiers déjà présents c'est à dire CTR0.0 et EID210.0

Document 1B la configuration de la maquette, éléments de logiciel

2.2 Eléments de solution

2.2.1 Analyse

Rappel

Chenillard sur feux arrière droit Si l'on souhaite définir l'état des sorties d'un module 4 sorties de puissance, il faut lui envoyer une trame de type « Input message » avec la fonction « Write Register » sur son registre d'accès au port de sortie « GPPIN » (d'après notice technique du circuit MCP25050 page 22).

En effet vu du module, il reçoit une trame de commande lui demandant de changer l'état de son registre de sortie.

Pour un « Input message » le registre concerné est le RXF1 ce qui conduit, pour le feu arrière droit à l'identificateur de base 0F 88 xx xx (voir tableau des identificateurs chapitre 1).

Les 3 bits de poids faibles doivent être obligatoirement à 0 (d'après notice technique du circuit MCP25050 page 22) et on choisit de mettre également 0 les autres bits indifferents, ce qui conduit en définitive à l'identificateur 0F880000 (seul 29 bits utiles).

Définition de la trame de commande (voir dans chapitre I de ce document la définition des différents champs de la trame destinée à cet exemple).

Remarques

- Dans la trame de commande, il y a trois paramètres définis dans la zone « data » de la trame :
 - Paramètre « addr » (au rang 0 des « data ») définit l'adresse registre concerné par l'écriture. Dans le cas présent, c'est GPPIN et son adresse est 1E (Page 37 notice MCP25050).
 - Paramètre « mask » (au rang 1 des « data ») permet de laisser inchangés certains bits du registre si ceux-ci ne doivent pas être affectés par l'opération d'écriture. Dans notre cas les sorties de puissance sont connectées sur les 4 bits de poids faibles du port. Les 4 bits de poids forts devront donc être masqués, soit la valeur du masque : 0F
 - Paramètre « value » (au rang 2 des « data ») permet de définir l'état des sorties non masquées. Dans notre cas, pour réaliser la fonction « chenillard », il faudra donner les valeurs successives 00, puis 01, puis 02, puis 04 et enfin 08.
- Suite à un « Input message », si le message a été bien reçu par son destinataire, celui-ci doit renvoyer une trame d'acquittement module dont l'identificateur est défini par TXID1. Dans le cas du module « feux arrière droit », l'identificateur de message d'acquittement sera 0F A0 00 00 (d'après tableau chapitre 1).

1.1 Organisation logicielle

1.1.1 Les fichiers de définitions

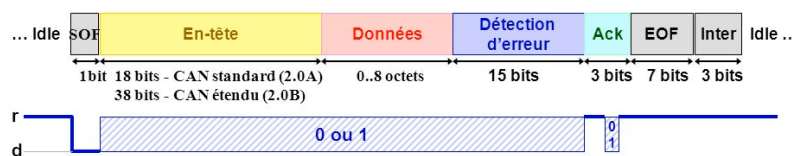
Les programmes fournis font appel à des fichiers de définitions :

- « Structures Donnees.h » définissant l'ensemble des structures de données utiles à une programmation structurée.
- « Mtr86.h » contient les prototypes des primitives de l'exécutif multi-tâche ainsi que des définitions de constantes. Il doit être impérativement placé en tête de tous les fichiers sources.
- « EID210.h » définissant les adresses d'accès aux différents éléments matériels implantés sur la carte processeur EID210.
- « vmd.h » définissant les éléments de l'application VMD.



CAN – format de la trame

- ☐ Start of Frame (SOF) / synchronisation
- ☐ En-tête (champ d'arbitrage / champ de contrôle)
- ☐ Champ de données
- ☐ Champ de contrôle des données/détection d'erreur
- ☐ Champ d'acquittement
- ☐ Fin de trame (EOF), Intermission (Inter)



Ecole des Mines de Nancy

9

CSSEA SI342b

Document 2 Structures de données les entêtes. h (header)

2.2.1 Analyse

A intervalles de temps réguliers, on interroge le module 8 entrées sur lequel est connecté le commodo lumière.

Définition de la trame image de la trame interrogative qui sera envoyée Dans ce cas, la trame envoyée par le contrôleur CAN (Circuit SJA 1000 sur carte CAN_PC104) sera vue par le récepteur (circuit MCP25050 sur module) comme un « Information Request message », avec la fonction « Read register » (voir documentation technique du MPC25025 pages 22).

D'après le tableau donné page 22 de la notice du MCP25050, l'identificateur lui-même contiendra l'adresse du registre lu. Cette adresse est placée sur les bits ID15 à ID8 de l'identificateur en mode étendu (bits qui seront réceptionnés et placés dans le registre RXBEIDR). Le registre concerné est « adresse 1Eh » (voir documentation technique du MPC25025 pages 37)... D'autre part, les trois bits de poids faibles de l'identificateur en mode étendu devront être positionnés à 1.

L'identificateur défini dans le chapitre 1 devra donc être complété comme suit (bits sont en compte) :

05 04 xx xx

D'après tableau Adresse registre Code fonction

05 04 1E 07

→ **Définition de variables structurées sous le modèle** Trame T_IRM_Commodo_Feux;

Trame destinée à l'interrogation du module 8 entrées sur lequel est connecté le commodo lumière Rem : La variable structurée T_IRM_Commodo_Feux comprend 4 octets utiles seulement, octet pour trame_info et 4 octets pour l'identificateur en mode étendu, octet pour l'adresse du registre concerné par la lecture.

→ **Accès et définition des différents éléments**

```
T_IRM_Commodo.trame_info.registre
T_IRM_Commodo.trame_info.champ
T_IRM_Commodo.trame_info.taille
T_IRM_Commodo.trame_info.type
T_IRM_Commodo.ident.extend_ident
T_IRM_Commodo.ident.ex
//1 c'est en fait la taille
```

Structure « T_IRM_Commodo » Initialise tous les bits à 0

On travaille en mode étendu

Type trame 1 octet de données

Pointeur, identificateur=0x05041E07

Pour l'adresse du commodo

Les valeurs des différents champs ont été déclarées dans le fichier CAN_VMD.h

Définition de la trame qui sera reçue en réponse D'après la définition des trames donnée en chapitre 1, une trame de réponse à une IRM a le même identificateur que l'IRM qui en a été à l'origine.

La réponse à un IRM (Information Request Message) est un OM (Output Message).

La seule originalité est que cette trame réponse comporte le paramètre « value » (image de la trame). Ce paramètre est l'image du port d'entrée. On récupère donc l'état des états binaires des commandes.

Document 3 Types de trames

Document 4 Gestion des tâches

Définition de la trame image de la trame interrogative qui sera envoyée

Dans ce cas, la trame envoyée par le contrôleur CAN (Circuit SJA1000 sur carte CAN_PC104) sera vue par le récepteur (circuit MCP25050 sur module) comme un "Information Request message", avec la fonction "Read register" (voir documentation technique du MPC25025 pages 22).

D'après le tableau donné page 22 de la notice du MCP25050, l'identificateur lui-même contiendra l'adresse du registre lu. Cette adresse est placée sur les bits ID15 à ID8 de l'identificateur en mode étendu (bits qui seront réceptionnés et placés dans le registre RXBEID8). Le registre concerné est l'adresse 1Eh " (voir documentation technique du MPC25025 pages 37)..

D'autre part, les trois bits de poids faibles de l'identificateur en mode étendu doivent être positionnés à 1.

L'identificateur défini dans le chapitre 1 devra donc être complété comme suit (les bits sont les bits en compte)



→ Définition de variables structurées sous le modèle

Trame T_IRM_Commodo_Feux;

// Trame destinée à l'interrogation du module 8 entrées, est contrôlée le commodo lumière

Rem: La variable structurée T_IRM_Commodo_Feux contient 5 octets utiles seulement, 1 octet pour trame_info et 4 octets pour l'identificateur en mode étendu. L'adresse du registre concerné par la lecture.

→ Accès et définition des différents éléments de la variable structurée "T_IRM_Commodo_Feux"

T_IRM_Commodo.trame_info.registre // Initialise tous les bits à 0

T_IRM_Commodo.trame_info.chtr // On travaille en mode étendu

T_IRM_Commodo.trame_info.type // Type trame

T_IRM_Commodo.trame_info.data // Il y aura 1 octet de données

T_IRM_Commodo.ident.ext // Identificateur ident=0x05041E07;

// c'est sur 29 bits pour l'adresse du commodo

Des labels définissant les bits des entrées ont été déclarés dans le fichier CAN_VMD.h

Définition de la trame image de la trame qui sera reçue en réponse

D'après la définition des données donnée en chapitre 1, une trame de réponse à une IRM a le même identificateur que la trame interrogative qui en a été à l'origine.

Vu du module 8 entrées, la réponse à un IRM (Information Request Message) est un OM (Output Message).

La trame de réponse interrogative origine est que cette trame réponse comporte le paramètre "value" (image de l'état du port d'entrée de la trame). Ce paramètre est l'image du port d'entrée. On récupère donc l'état

Accès aux données des commandes

Le paramètre "value" de la trame réponse, récupéré dans la donnée de rang 0 est un octet image des entrées.

Les différents bits de cet octet sont extraits individuellement grâce à une structure de données définie dans le fichier CAN_VMD.

Question 1

Donner pour chaque carte les différents composants.

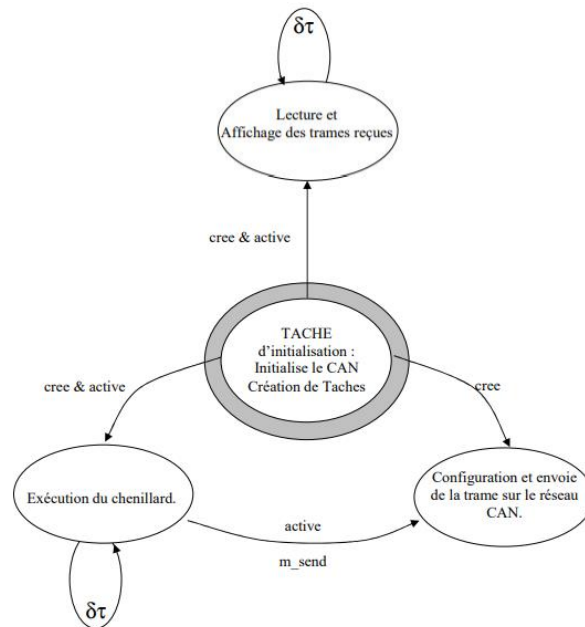
Question 2 :

En analysant le doc 2 dites quel est le rôle des entêtes. h introduites dans chacune des structures ?

Question 3 :

En analysant le doc 3, spécimen, donner les différents Types de trames comment les prioriser, expliquer le mode de fonctionnement ?

Diagramme d'états du système de contrôle



Document 4 Gestion des tâches pour un feu

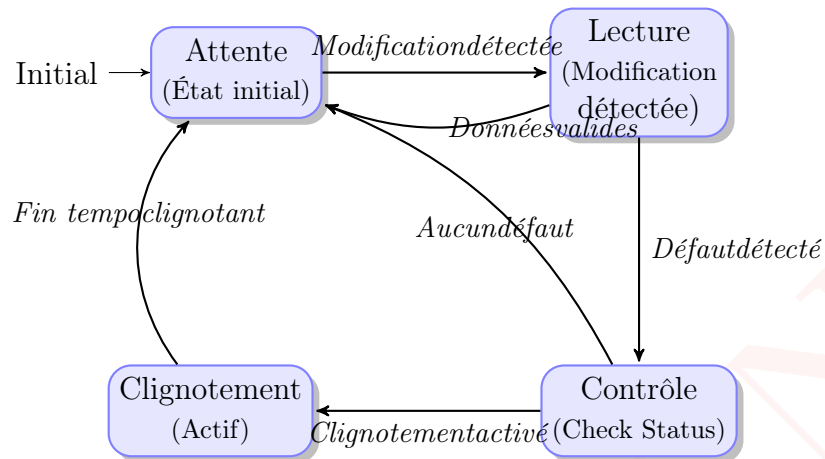
Question 4 :

En analysant le doc 4, décrire le mécanisme de gestion des tâches pour gestion des feux ?

PROGRAMMATION

```

1 #include <stdio.h>
2 #include "mtr86.h"
3 #include "Structures donnees.h"
4 #include "vmd.h"
5 #include "eid210.h"
6
7 //Selection simple d'un bloc optique
8 #define Feux_Selection_Ident T_IN_FEUX
9
10 //Prototype de Taches
11 TACHE init(void);
12 TACHE LectureTrame(void);
13 TACHE chenillard(void);
14 TACHE EcrireFeux();
  
```



Légende :

- États rectangulaires arrondis
- Transitions en italique
- Hiérarchie verticale

FIGURE 2 – Diagramme d'états amélioré du système de contrôle avec gestion des erreurs et temporisation

```

15 //Tache d'initialisation
16
17 TACHE init(void)
  
```

Listing 1 – "Chenillard sur le Feux Arriere Droit"

Références

- [1] https://www.irit.fr/Zoubir.Mammeri/Cours/Introduction_CAN.pdf

Caractéristiques techniques - EID 004 000 - Contrôleur de BUS CAN



Fonction	Contrôle du bus CAN.
Environnement	Implantation sur la carte EID 210 000 ou toutes cartes compatibles PC 104.
Contrôleur CAN	SJA 1000 Philips.
Interface	De type optoélectronique.
Vitesse de communication	Jusqu'à 1 Mbits/s.

Caractéristiques techniques - EID 050 000 - Carte BUS CAN 8 entrées TOR

Fonction	Permet de générer des commandes utilisateurs sur le bus CAN.
Environnement	Raccordement directement sur le bus CAN.
Alimentation	2 connecteurs 0/12 V (1 arrivée, 1 reprise). Régulateur 5 V sur la carte.
Microcontrôleur	CAN Expander MCP25020.
Entrées TOR	4 poussoirs, 4 microswitchs.
Entrée analogique	1 potentiomètre sur entrée analogique 0/5 V et convertisseur 10 bits.
Connecteur 10 pts	8 entrées TOR, 1 entrée analogique, 1 point commun.
Visualisation	8 LED de visualisation des entrées.



Caractéristiques techniques - EID 051 000 - Carte BUS CAN 4 sorties TOR de puissance



Fonction	Permet d'actionner des lampes 12 V/2 A par le biais du bus CAN.
Environnement	Raccordement directement sur le bus CAN.
Alimentation	2 connecteurs 0/12 V (1 arrivée, 1 reprise). Régulateur 5 V sur la carte.
Microcontrôleur	CAN Expander MCP25050.
Sorties TOR	4 sorties TOR de puissance 12 V/2 A avec contrôle courant.
Visualisation	4 LED de visualisation des états, raccordées sur les sorties de puissance.
Simulation de panne	1 microswitch de simulation de rupture de filament.

Caractéristiques techniques - EID 052 000 - Carte de commande et régulation de vitesse moteur

Fonction	Permet de piloter un moteur (essuie-glace avec asservissement de vitesse). Gestion des sécurités de fin de course.
Environnement	Raccordement directement sur le bus CAN.
Alimentation	2 connecteurs 0/12 V (1 arrivée, 1 reprise). Régulateur 5 V sur la carte.
Microcontrôleur	CAN Expander MCP25050.
Etage de puissance	Hacheur 4 quadrants L6203. PWM 1A sur 12 Vdc.
Entrée codeur	1 entrée codeur
Entrées TOR	2 entrées TOR fins de course. 1 entrée surcourse. 3 microswitchs de simulation de fins de course et de surcourse.
Entrée analogique	1 potentiomètre.

